Сборка пользовательского сценария

Валерий Линьков
Эксперт по проектированию облачной инфраструктуры и систем безопасности

Проверка связи

Если у вас нет звука:

- убедитесь, что на вашем устройстве и на колонках включён звук
- обновите страницу вебинара (или закройте страницу и заново присоединитесь к вебинару)
- откройте вебинар в другом браузере
- перезагрузите компьютер (ноутбук) и заново попытайтесь зайти

Поставьте в чат:

+ если меня видно и слышно

— если нет

Рекомендации

Если смотрите с компьютера

- Используйте браузеры Google Chrome или Microsoft Edge
- Если есть проблемы с изображением или звуком, обновите страницу — **F5**

Если смотрите с мобильного телефона или планшета

- Перейдите с мобильного интернет-соединения на **Wi-Fi**
- Если есть проблемы с изображением или звуком, перезапустите приложение на телефоне
- Предварительно проверьте, подходит ли ваше устройство для подключения к вебинару, по ссылке

Валерий Линьков

О спикере:

- Проектирую облачную инфраструктуру и системы безопасности
- Работаю с архитектурой, аутентификацией и управлением доступами, а также с инженерными процессами и обучением команд
- Занимаюсь архитектурными аудитами, консалтингом и technical enablement: объясняю решения, выстраиваю подходы и помогаю командам работать с архитектурой и безопасностью осознанно

Цели занятия

- Понять, как собрать рабочий сценарий сервиса из отдельных компонентов
- Выяснить, как связывать компоненты сервиса в единый пользовательский сценарий
- Узнать, как анализировать зависимости между компонентами
- Разобраться, как связаны компоненты, какую роль играет ИИ в сборке и как проводить проверку перед запуском

План занятия

1. Ключевые компоненты
2. Типичные директории
3. Особенности ИИ
4. Прогон
5.
6.

Вспоминаем прошлое занятие

Вопрос: Что такое «пользовательский pipeline»?

Вспоминаем прошлое занятие

Вопрос: Что такое «пользовательский pipeline»?

Ответ: Пользовательский pipeline — это последовательность этапов обработки запроса пользователя (UI → API → бизнес-логика → LLM → постобработка → UI), через которые проходит данные, чтобы сформировать итоговый ответ сервиса.

Ключевые компоненты

От pipeline к сервису

Pipeline — это логическая схема.

Сценарий сервиса — это:

- файлы
- модули
- конфигурации
- интеграции

То есть реальная реализация pipeline

Сборка сценария

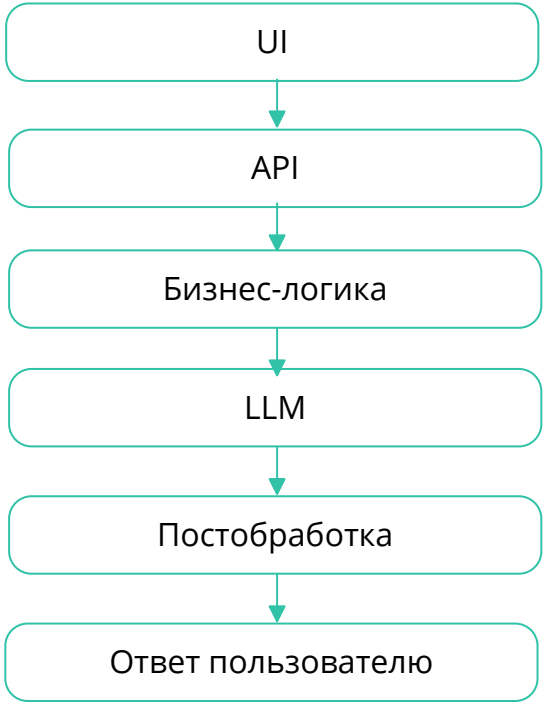
Сборка сценария — это:

1. Соединение компонентов
2. Настройка конфигураций
3. Проверка интеграций
4. Тестовый запуск

Если один элемент выпадает — сценарий не работает

Типичный pipeline LLM-сервиса

Пользовательский запрос проходит через несколько слоёв:



Компоненты сервиса

Минимальный набор:

- UI или интерфейс
- Эндпоинт API
- Бизнес-логика
- Интеграция с LLM
- Постобработка ответа
- Логирование

Каждый компонент выполняет свою роль

Ваши вопросы

Типичные директории

Структура проекта

Обычно структура проекта выглядит так:

- /api
- /services
- /llm
- /config
- /tests

Структура проекта

Связка происходит через:

- импорты
- конфиги
- переменные окружения

/api

Отвечает за точки входа сервиса: принимает запросы пользователя (HTTP, REST, webhook), валидирует данные и передаёт их в бизнес-логику

/services

Содержит бизнес-логику приложения: обработку запроса, координацию компонентов и формирование задачи для LLM

/llm

Отвечает за взаимодействие с языковой моделью: формирование промптов, отправку запроса в модель и получение ответа

/config

Хранит конфигурации сервиса: параметры работы, настройки моделей, ключи API, переменные окружения

/tests

Содержит тесты для проверки работы компонентов и пользовательского сценария перед запуском

Точки интеграции

Наиболее чувствительные места:

- API → бизнес-логика
- Логика → LLM
- LLM → постобработка
- Конфигурации и ключи

Большинство ошибок появляется именно здесь

Паттерны связки компонентов

Распространённые архитектурные подходы:

- **Pipeline Workflow** — последовательные этапы, каждый независимо масштабируется
- **Orchestrator-Worker** — центральный координатор делегирует задачи специализированным воркерам
- **Adapter Integration** — переходный слой для связки с legacy-системами
- **Hybrid** — монолит для критичных операций + микросервисы для вспомогательных

Ваши вопросы

Особенности ИИ

Роль ИИ при сборке

ИИ может помочь:

- Создать черновики модулей
- Предложить структуру проекта
- Написать конфигурации
- Подсказать рефакторинг

Но есть нюанс

Ограничения ИИ

ИИ не знает:

- инфраструктуру проекта
- реальные ключи и настройки
- зависимости между сервисами
- состояние среды выполнения

Поэтому финальная сборка — задача разработчика.

ИИ — помощник, не архитектор

Примеры промптов для ИИ

Для генерации черновика модуля:

```
Сгенерируи черновик модуля services/chat.py для Python-проекта.
Модуль должен принимать валидированныи запрос от /api и формировать
вызов к LLM-клиенту. Не добавляи реальные ключи
```

Примеры промптов для ИИ

Для создания конфигурации:

```
Создаи пример фаила config/llm.yaml для сервиса с вызовом LLM API.
Включи поля: base_url, timeout, retries. Значения оставь заглушками
```

Что проверить в ответе ИИ

Обязательная ручная проверка:

- Структура импортов соответствует вашей архитектуре
- Контракты (форматы входа/выхода) не нарушены
- Нет фейковых ключей или URL в конфигах
- Обработка ошибок соответствует стандартам проекта

Результат ИИ — это черновик, не готовый код

Ваши вопросы

Прогон

Что проверить перед запуском

Перед первым запуском нужно проверить:

- Конфигурации
- Ключи API
- Форматы данных
- Обработку ошибок
- Логирование

Это минимальный чек-лист

Чек-лист ручных проверок

Что проверить	Как проверить	Чем подтвердить
Конфигурации	Сверить /config с ТЗ	Скриншот конфига
Ключи API	Проверить env/secrets	Вывод env
Формат данных	Тестовый запрос curl	Пример запроса/ответа
Обработка ошибок	Битый JSON, пустое поле	Логи с ошибкой

Тестовый прогон сценария

Первый запуск — это тестовый пользовательский запрос.

Цель:

Проверить полный путь запроса

Проверка контрактов

Контракт определяет:

- Формат запроса
- Формат ответа
- Правила обработки ошибок

Проверка контрактов

Типичные проблемы:

- Неправильный JSON
- Неожиданный ответ LLM
- Пустые поля

Типы тестирования LLM-сервисов

Специфика тестирования LLM:

- **Функциональное** — проверка валидности промптов, корректности ответов, обработка edge cases
- **Качество ответов** — фактическая корректность, тон, грамматика, паттерны
- **Контекст** — многоходовые диалоги, память контекста, логическая связность
- **Безопасность** — утечки PII, prompt injection, обработка чувствительных данных

Анализ логов

После тестового запуска проверяем:

- Ошибки API
- Ошибки парсинга
- Задержки запросов
- Лимиты LLM

Логи показывают, где именно ломается сценарий

Мини-кейс: анализ ошибки

Пример лога:

```
POST /api/chat 500
Error: JSONDecodeError: Expecting ',' delimiter
Request body: {"message": "Привет" "user_id": 123}
...
LLM response parse error: missing field "answer"
```

Вопросы:

- На каком этапе произошла первая ошибка?
- Какую проверку из чек-листа стоило сделать раньше?

Инструменты мониторинга и наблюдаемости

Что важно отслеживать:

- **Трейсинг** — полный путь запроса от входа до выхода
- **Метрики токенов** — input/output token count для контроля затрат
- **Латентность API** — время ответа каждого компонента
- **Использование ресурсов** — CPU, GPU, память

Структурированное логирование с метаданными — основа наблюдаемости

Вопрос

Вопрос: напишите в чат три обязательные проверки, без которых вы не запустите свой LLM-сервис.

Примеры:

- Формат JSON-запроса к API проверен руками через Postman
- Лимиты LLM и таймауты заданы и проверены в конфиге
- Логирование по всему пути запроса включено и протестировано

Ваши вопросы

Итоги

Подведём итоги

Сборка сценария включает :

- → Связку компонентов
- → Проверку интеграции
- → Тестирование pipeline
- → Проверку корректности ответа

Подведём итоги

- Сквозной сценарий сервиса — это конкретные файлы, модули, конфигурации и интеграции, реализующие логическую схему pipeline
- Основные источники ошибок — точки интеграции: API → бизнес-логика → LLM → постобработка, конфигурации и ключи
- ИИ помогает как ассистент: генерирует черновики модулей и конфигов, но любой его код требует обязательной ручной проверки
- Перед запуском обязателен чек-лист: конфиги, ключи API, форматы данных, обработка ошибок, логирование

Подведём итоги

- Первый запуск — это тестовый прогон полного пути запроса с проверкой контрактов и анализом логов
- Логи, метрики и трейсинг — главный инструмент понимания, где ломается сценарий и как он ведёт себя в проде
- Инженер фиксирует свои обязательные проверки и использует ИИ как инструмент ускорения, а не источник готового прод-кода

Сборка пользовательского сценария

Валерий Линьков
Эксперт по проектированию облачной инфраструктуры и систем безопасности